

Container Registry: Distribución de Imágenes Docker

Nombre del autor

 **Tiempo estimado: 10 min**

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Amabilidad	Se aplica en los entornos DevOps donde el trabajo colaborativo es constante: compartir imágenes, revisar configuraciones de Docker Compose de compañeros o reportar errores en contenedores. La forma en que se hace una solicitud de ayuda impacta directamente en la disposición del equipo. La amabilidad construye la confianza necesaria para que un equipo de desarrollo e infraestructura funcione de manera efectiva.

1.2 Competencia(s)

Tipo de Competencia	
Competencia General	Analiza conceptos fundamentales de Docker para el empaquetado y despliegue de aplicaciones.
Competencia Específica	Utiliza un Container Registry para versionar, almacenar y distribuir imágenes Docker en flujos de trabajo colaborativos y de integración continua.

2. Palabras Clave

Container Registry : Servicio centralizado para almacenar, versionar y distribuir imágenes Docker.

Imagen Docker : Plantilla inmutable que contiene el código, las dependencias y la configuración necesaria para ejecutar un contenedor.

Tag : Etiqueta que identifica una versión específica de una imagen Docker (por ejemplo, `app:1.0.0`).

Push / Pull: Operaciones para subir una imagen al registry y descargarla desde él, respectivamente.

3. Introducción

En el desarrollo de software moderno, Docker permite empaquetar aplicaciones en contenedores portables y reproducibles. Pero para que esos contenedores sean verdaderamente útiles en equipos y en producción, es necesario un lugar donde almacenarlos y distribuirlos: ese lugar es el Container Registry. Un registry cumple para las imágenes Docker el mismo rol que GitHub cumple para el código fuente: es el punto centralizado desde donde cualquier servidor o miembro del equipo puede obtener la versión correcta del software.

4. Contenidos

¿Qué es un Container Registry?

Un Container Registry es un servicio para almacenar, versionar y distribuir imágenes Docker. Permite que los equipos compartan imágenes de forma estandarizada, que los pipelines de CI/CD descarguen imágenes automáticamente y que los servidores de producción ejecuten siempre la versión correcta de una aplicación.

Principales Registries Disponibles

Registry	URL base	Características principales
Docker Hub	hub.docker.com	Público y gratuito; el más usado para imágenes open source
GitHub Container Registry (GHCR)	ghcr.io	Integrado con GitHub Actions; ideal para proyectos en GitHub
AWS ECR	*.amazonaws.com	Privado; optimizado para despliegues en AWS
Google GCR	gcr.io	Privado; optimizado para despliegues en GCP

Registry privado	registry:2	Imagen oficial de Docker para alojar un registry propio
------------------	------------	---

Tabla 1 - Principales Registries Disponibles

Flujo de Trabajo con Docker Hub

El flujo básico para publicar una imagen en Docker Hub sigue estos pasos:

1. Iniciar sesión: `docker login`
2. Etiquetar la imagen con el nombre de usuario: `docker tag mi-app:1.0.0 usuario/mi-app:1.0.0`
3. Subir la imagen al registry: `docker push usuario/mi-app:1.0.0`
4. Desde cualquier servidor o equipo: `docker pull usuario/mi-app:1.0.0`

Flujo de Trabajo con GitHub Container Registry (GHCR)

GHCR es el registry más utilizado en proyectos colaborativos modernos porque se integra directamente con los repositorios y los flujos de CI/CD de GitHub:

1. Autenticarse con un token de GitHub: `echo $GITHUB_TOKEN | docker login ghcr.io -u USERNAME --password-stdin`
2. Etiquetar la imagen para GHCR: `docker tag mi-app:1.0.0 ghcr.io/organizacion/mi-app:1.0.0`
3. Subir la imagen: `docker push ghcr.io/organizacion/mi-app:1.0.0`

Tabla 2 — Comparativa de Flujos de Trabajo

Paso	Docker Hub	GitHub Container Registry
Autenticación	<code>docker login</code>	Token de GitHub

Formato del tag	usuario/imagen:version	ghcr.io/org/imagen:version
Integración CI/CD	Manual o por workflows	Nativa con GitHub Actions
Visibilidad	Pública por defecto	Controlada por permisos del repositorio

Tabla 2 - Comparativa de Flujos de Trabajo Docker HubGitHub vs Container Registry

Caso de estudio

Un equipo de tres desarrolladores trabajaba en una API REST para una aplicación de gestión de eventos. Durante meses compartieron las imágenes Docker enviándose archivos `.tar` por correo o copiándolos manualmente entre servidores. El proceso era lento, propenso a errores de versión y completamente imposible de automatizar.

El problema se hizo crítico cuando el servidor de staging ejecutaba la versión `0.8.0` de la imagen mientras producción tenía la `0.7.2`, pero nadie sabía con certeza qué diferencias había entre ambas porque no existía un historial confiable.

El equipo implementó GitHub Container Registry conectado a su repositorio. Configuraron un workflow de GitHub Actions que, al hacer merge a la rama principal, construía automáticamente la imagen, la etiquetaba con el número de versión y la subía al registry. A partir de ese momento:

- Cualquier servidor podía ejecutar `docker pull ghcr.io/equipo/eventos-api:1.2.0` y obtener exactamente la versión correcta.
- El historial de imágenes quedaba vinculado al historial de commits.
- Los despliegues pasaron de tomar 40 minutos a tomar 8 minutos.

La adopción del registry no solo resolvió el problema técnico, también mejoró la comunicación del equipo: todos hablaban el mismo idioma al referenciar versiones específicas de la aplicación.

5. Conclusiones

Un Container Registry es un componente esencial en cualquier flujo DevOps moderno: sin él, distribuir imágenes entre entornos y equipos se vuelve manual, propensa a errores y difícil de rastrear.

La elección del registry depende del contexto del proyecto: Docker Hub es ideal para proyectos públicos, mientras que GHCR o los registries de nube son más apropiados para proyectos privados o con integración continua.

Etiquetar correctamente las imágenes con versiones semánticas permite mantener un historial claro y reproducible de los despliegues realizados.

6. Referencias

1. Docker Inc. (2024). *Docker Hub Documentation*. <https://docs.docker.com/docker-hub/>
2. GitHub Docs. (2024). *Working with the Container Registry*. <https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry>